MANUAL TEST GUIDE · MODULE 05 OF 11

Equipment Operations (CECS) Functional-Flow Test Walkthroughs

Scenario-driven QA guide for the **logistics.equipment_operations** physical-truth layer — the mode-agnostic equipment registry, the `equipment.usage` binding every downstream module references, append-only seal & movement logs, and the load-calculation + 3D-viewer engines. Behaviour is **hook-driven**: create / update / delete an operational record and the pre/post hooks are the code under test.

> **How to use:** Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives a precondition, an action (a record op or a command), the expected result, and a ✔ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip an **append-only** or validation guard to confirm it holds.

PREPARED FOR	SOURCE INPUTS	DEMO LANE	DATE
Manual QA / UAT	CECS BRS (BR-CECS/EQ/USG/SEAL/EVT/CALC) · impl docs · demo use-case	Globex INMUM → SGSIN	15 July 2026

SECTION 01

01 What Equipment Operations does & how to read this guide

CECS (Centralized Equipment Control System) is the **mode-agnostic physical truth layer**. One model — `logistics.equipment` — holds every real container, truck, wagon or ULD; mode is read through `equipment_type_id.transport_mode_id`, never a per-mode subclass. The load-bearing record is `logistics.equipment.usage` — the "stay" that binds a physical unit to an operation for a time window, referenced by Booking, Shipments, Consolidation, Documentation, Ops Execution & Tracking (BR-CECS-01/25/60).

7	18	3	3
COMMANDS TO EXERCISE	LIFECYCLE HOOKS	APPEND-ONLY LOGS	TEST SCENARIOS

The two derived state machines you will observe

Equipment status and usage status are both **derived**, never free-typed. Usage status flips from movement events once any event exists on the usage (BR-CECS-24/43).

```
equipment.current_status : Available → Reserved → In-Use → Available (loop) | Under-Repair · Damaged · Retired (terminal)
usage.status             : planned → reserved → active → completed | cancelled  (reserved on allocate; derived after 1st event)
event-code → usage       : loaded → active · delivered → completed · cancelled → cancelled (BR-EVT-10, hardcoded Phase 1)
```

The physical-truth layer everyone else reads

```
logistics.equipment ──► equipment.usage ──► { seal.application* · movement.event* }  (* = append-only)
    equipment.usage is referenced by: Booking · Shipments · Consolidation · Documentation · Ops Execution · Tracking
    one usage binds one shipment leg; consolidation lets one usage host many shipments (BR-CECS-05/26)
```

Four classifications set at record creation

LEASE_TYPE · SEAL_SLOT

- **lease_type** — owned · leased · chartered · carrier_supplied
- **seal_slot** — primary · secondary · customs · redundant (label only, no uniqueness)

EVENT SOURCE · LOAD RESULT_STATUS

- **source** — manual (Phase 1) · edi · carrier_api · iot (forward-compat)
- **result_status** — single_unit_fit · multi_unit_fit · no_fit_oversize · no_fit_capacity · no_fit_compatibility

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Register & bind	Register a mode-agnostic unit, allocate a usage on the INMUM→SGSIN leg, seal it, drive it to completion via movement events — every validator + status derivation fires	`allocate.usage`, `apply.seal`, `log.movement.event`	§02
B — Append-only integrity	The signature guard: every edit/delete against a seal, movement event, usage-with-history, or load-calc is **refused** with a specific ValueError	ede.update / ede.delete, `remove.seal`	§03
C — Load calc + 3D	Run the deterministic three-check load engine (audit row), then open the equipment-type 3D viewer client action	`calculate.load`, `equipment.3d_viewer`	§04

Demo fixtures you start from (loaded by `--with-demo=all`)

Forwarder **Acme Forwarding — Mumbai** runs its own fleet on the INMUM→SGSIN lane; custodian is `base.default_organization`. Seven equipment rows ship — `MSKU4200001/02` (40ft dry), `MSKU4250001` (40ft reefer), `MSKU2200001` (20ft, In-Use), `GJ01-AB-1234` (tractor), `CHAS-40-001` (chassis), `TCLU5500001` (carrier-supplied). Operations: `USG-DEMO-0001` (Active), `USG-DEMO-0002` (Reserved), seal `BOLT-DEMO-778812`, GATE_OUT/GATE_IN events, and a single-unit-fit load-calc. All CECS taxonomy is referenced from the `logistics.masters` seed, never recreated.

SECTION 02 · SCENARIO A

02 Register a unit & bind it to the export leg

Goal: take a mode-agnostic equipment row from registration → usage allocation on the INMUM→SGSIN leg → seal → completion, confirming each hook fires. **Role:** CECS dispatcher. **Start state:** the seeded Acme fleet + CECS masters are loaded; pick an **Available** 40ft dry unit.

#	Action	Expected result	✔ Checkpoint	Trace
1	Create a `logistics.equipment` row with a valid `equipment_type_id` and identifier `MSKU4200099`.	Pre-create hook validates the identifier against the type's identifier-type `pattern` (BR-EQ-01), **denormalises** `equipment_category_id` + `transport_mode_id` from the type, and defaults `current_status_id` to the first `is_available_state=true` row.	**Row saved; category & mode auto-filled; status = Available.**	BR-EQ-01 BR-CECS-01
2	Retry step 1 with a malformed identifier (e.g. `not-a-box`) against a type whose identifier-type carries a pattern.	Rejected: `equipment_identifier '…' does not match the format required by identifier-type '…' (pattern: …) — BR-EQ-01`.	**Bad identifier blocked.**	BR-EQ-01
3	Run `equipment-operations.allocate.usage` binding that equipment to a window on the export leg (origin/dest facilities, `planned_start_at` / `planned_end_at`).	Usage created; `usage_code` stamped from the `USG` sequence; `status_id` defaults to **Reserved**; the equipment's status is snapshotted into `previous_equipment_status_id`.	**`USG-…` code present; status Reserved.**	BR-USG-05 BR-CECS-25
4	Re-read the parent `logistics.equipment` row.	Post-create hook flipped `current_status_id` → **Reserved** and emitted `equipment.usage.allocated`.	**Equipment now Reserved; event fired.**	BR-USG-05
5	Run `equipment-operations.apply.seal` — an initial bolt seal, `seal_slot=primary`, no change reason (initial application is exempt).	Seal row appended on the usage; `organization_id` denormalised from the usage; `equipment.seal.applied` emitted.	**Seal on usage; initial needs no reason.**	BR-SEAL-04 BR-CECS-30
6	Run `equipment-operations.log.movement.event` for a `loaded` then a `delivered` event-type.	Usage status is **derived** from the stream: loaded → **active** (stamps `actual_start_at`), delivered → **completed** (stamps `actual_end_at`); on delivered the equipment returns to **Available**; `equipment.usage.completed` emitted.	**Status walked to completed; not hand-typed.**	BR-EVT-06 BR-CECS-43

✔ Scenario A pass criteria

A single physical unit moved registry → Reserved → Active → Completed with the seal and both actual timestamps written entirely by hooks. Usage status was never set by hand — it fell out of the event stream (BR-CECS-04/24/43).

⚠ Overlap & idempotency gotchas

Allocating a **second** usage on the same equipment with an overlapping window is refused — `window overlap with existing usage '…' (…); overbook disabled (BR-USG-02..04)` — unless it is within `EQUIPMENT_USAGE_OVERLAP_GRACE_MINUTES` (30) or `equipment.allow_overbook=true` for a Reserved-vs-Reserved case. Re-logging the same event within `equipment.event.idempotency_window_seconds` (60) is **silently dropped** and returns the existing event id (BR-EVT-03).

SECTION 03 · SCENARIO B — THE APPEND-ONLY DEEP-DIVE

03 Append-only integrity — every edit & delete must be refused

This is the module's signature guard. Seal applications, movement events and load-calc rows are **immutable audit anchors** — history is preserved forever (BR-CECS-03/04/33/71). A usage that carries any history cannot be hard-deleted; it is cancelled. Each row below trips a specific `ValueError` from a pre-hook — quote the message verbatim in your test evidence.

#	Action (record op / command)	Expected refusal — real message	✔	Trace
1	Edit any non-removal field (e.g. `seal_number`) on an existing `equipment.seal.application`.	"seal applications are append-only — fields […] cannot be edited; only removal-event fields […] may be set (BR-SEAL-02)".	✗	BR-SEAL-02
2	Hard-**delete** a seal application.	"seal applications are append-only and cannot be deleted; record a removal event with `equipment-operations.remove.seal` instead (BR-SEAL-02)".	✗	BR-SEAL-02
3	Call `remove.seal` without `removed_reason_id`.	"remove.seal requires removed_reason_id". (With a reason it succeeds, writing only the removal fields.)	✗	BR-SEAL-02
4	Call `remove.seal` again on a seal that already has `removed_at` set.	"seal already closed; re-opening is not allowed — apply a new seal row instead (BR-SEAL-03)".	✗	BR-SEAL-03
5	Edit a field on an existing `equipment.movement.event`.	"movement events are append-only — record a compensating event instead of editing (BR-EVT-02)".	✗	BR-EVT-02
6	Hard-**delete** a movement event.	"movement events are append-only and cannot be deleted (BR-EVT-02)".	✗	BR-EVT-02
7	Append a movement event to a **cancelled** usage.	"cannot append movement events to a cancelled usage (BR-EVT-01 / BR-EVT-09)".	✗	BR-EVT-01
8	Manually write `status_id` on a usage that already has ≥1 movement event.	"manual status writes rejected — at least one movement event exists on this usage; status is derived from the event stream (BR-USG-06)".	✗	BR-USG-06
9	Directly write `responsibility_party_id` on a usage via `ede.update`.	"responsibility_party_id is read-only outside the custody_transfer event flow (BR-USG-09)".	✗	BR-USG-09
10	Hard-**delete** a usage that carries seal or event history.	"usage '…' carries logistics.equipment.… history and cannot be hard-deleted; cancel it instead (BR-USG-07)".	✗	BR-USG-07
11	Edit — then delete — a `logistics.load.calculation` row.	Edit: "load-calc rows are immutable audit anchors; create a new calculation instead of editing (BR-CALC-08)". Delete: "load-calc rows are immutable audit anchors and cannot be deleted".	✗	BR-CALC-08

⚠ The one field you *may* toggle

Soft-archive via the `active` boolean is the **only** permitted write on these logs — the update hooks whitelist `active` so an admin can hide a mistaken row without rewriting the stream. Seal removal additionally permits the five removal-event fields (`removed_at` , `removed_by_*` , `removed_reason_id` , `removed_notes`). Any other key in the payload trips the guard.

✔ Scenario B pass criteria

All eleven attempts refused with the exact messages above; the seed rows (seal `BOLT-DEMO-778812` , the two GATE events, the demo load-calc) are unchanged afterward. Corrections happen by **appending** — a removal event + successor seal, a compensating movement event, or a fresh load-calc — never by mutation (BR-CECS-33).

SECTION 04 · SCENARIO C

04 Load calculation & the equipment-type 3D viewer

Goal: run the deterministic Phase-1 load engine — turn cargo lines into an equipment-type plan and persist an immutable audit row — then open the 3D viewer to inspect the recommended type to scale. **Role:** planner. **Start:** CECS masters + `equipment_render_config` seed loaded.

#	Action	Expected result	✔ Checkpoint	Trace
1	Run `equipment-operations.calculate.load` with an `organization_id` and a non-empty `cargo_lines` list that fits one 40ft.	Engine runs the three checks (compatibility / volume / weight / floor-area); persists one `logistics.load.calculation` audit row; stamps `algorithm_version=phase1.v1` and a `LCALC-…` code; returns `plan_json` + `result_status=single_unit_fit`; emits `equipment.load.calculation.completed`.	`plan_json=[{…,qty:1}]`; **audit row saved.**	BR-CALC-06 BR-CALC-07
2	Call it with no `organization_id`, then with an empty `cargo_lines`.	Rejected: "calculate.load requires organization_id in payload"; then "calculate.load requires a non-empty `cargo_lines` list".	**Both guards refuse.**	BR-CALC-06
3	Feed cargo whose largest dimension exceeds every candidate type's internal size.	`result_status=no_fit_oversize`, empty `plan_json`. Over-capacity-but-fits cargo instead returns `multi_unit_fit` with a computed `qty`; nothing at all returns `no_fit_capacity`.	**Correct no-fit reason returned.**	BR-CALC-03/04
4	Confirm the persisted row is immutable (cross-ref §03 row 11).	Any edit/delete of the audit row is refused — the calc is re-run, never patched.	**Audit anchor holds.**	BR-CALC-08
5	Open an `equipment.type.master` row/form and invoke **3D View**.	The `equipment.action_3d_viewer` client action (`client_component=equipment.3d_viewer`, `target=dialog`) opens the R3F viewer as a dialog rendering only that type to scale (orbit/zoom only).	**3D dialog opens for the type.**	Enh 03
6	Compare the shape of a dry vs a reefer subtype.	Shape resolves `subcategory.render_archetype → category.render_archetype → generic`: container category → `box`; reefer subcategory → `box` / `variant=reefer`; only `supports_3d_view=true` categories are viewable.	**Archetype/variant resolve as seeded.**	Enh 03

THE PHASE-1 LOAD ENGINE (DETERMINISTIC)

- Checks: compatibility gate · volume fit · weight fit · floor-area (only when non-stackable present)
- Single-unit tie-break = **smallest usable volume** (BR-CALC-03); multi-unit picks smallest total volume
- hazmat / temperature gates are **pass-through** in Phase 1 (schema enhancement pending — recorded deviation, BR-CALC-05)

3D RENDER CONFIG (EXTEND_MODEL)

- Render columns live on `category.master` / `subcategory.master` via `@api.extend_model` — masters stay owned by `logistics.masters`
- `render_archetype`: box · tank · uld · vehicle · wagon · trailer · chassis · generic
- `render_variant`: default · dry · reefer · open_top · flat_rack · pallet

✔ Full-module pass criteria

All three scenarios green: the lifecycle rides hooks end-to-end (A), every append-only edit/delete is refused with its real message (B), and the load engine produces an immutable audited plan whose recommended type renders to scale in the 3D dialog (C). Because this is the physical-truth layer, its `equipment.usage` rows are what Booking, Shipments, Consolidation, Documentation, Ops Execution and Tracking bind to — verified in those modules' own guides.

Grounded against real code: 7 `@api.on_command` handlers + 18 `@api.on_hook` across `equipment.py`, `usage.py`, `seal_application.py`, `movement_event.py`, `load_calculation.py`. Note: the impl doc lists Enhancement 03 (3D viewer) as Not-Started, but the action, render extension and R3F frontend are present in the module — treat §04 rows 5–6 as built.